
Data Wrangling and Cleaning

...

POLI3148 Data Science in Politics and Public Administration

Dr. Chen Haohan



Agenda

- Logistics
 - Feedback for the first two in-class exercises will be with you by next Monday
 - Attendance and participation self-report
 - Start to think about your final project
 - Data Wrangling and Cleaning
 - In-class exercise 3
- 



Path towards a Successful Data Science Project

In: Questions and data

- Your curiosity
- Existing puzzles and theories (from literature, experience, narrative, ...)
- Data science tools: data and models

⇒

Out: Discovery

- Accurately describe what things are
 - Insightfully explain why things look that way
 - Convincingly discuss why the audience should care about this discovery
- 

Effective Learning

- This course focuses on helping you develop a successful data science project
 - The focus is establish a data scientist' mindset (the “big picture”)
 - You will work on the technical details
 - Self-study (use recommended resources and beyond)
 - Talk to us
-

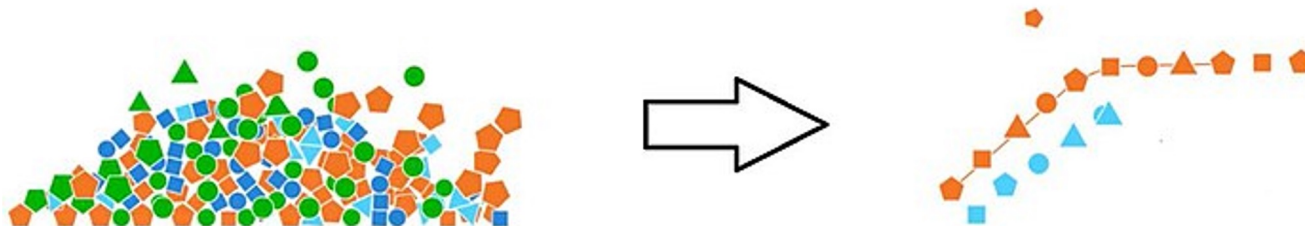


Data Wrangling and Cleaning



The Big Picture: Why Bother?

- Data are MESSY: You seldom starts with what you need to answer your question
- Data analysis requires CLEAN data
 - Filter the subset of data you need
 - Create indicators you care
 - Transform into formats that your data analysis tools can work with



Readings: Kaggle “Pandas” and “Data Cleaning”

Pandas

- Creating, reading, and writing
 - Indexing, selecting & assigning
 - Summary functions and maps
 - Grouping and sorting
 - Data types and missing values
 - Renaming and combining
-

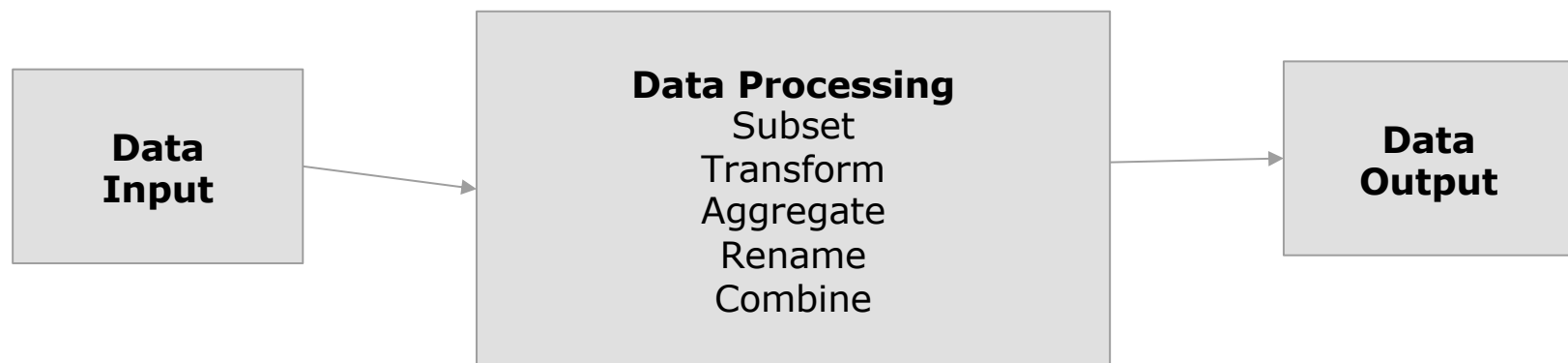
Readings: Kaggle “Pandas” and “Data Cleaning”

Data Cleaning

- Handling missing values
 - Scaling and normalization
 - Parsing dates
 - Character encodings
 - Inconsistent data entry
-

Map skills into our data science workflow

- Data Input: Read raw data, get data overview
- Data processing
 - Subset
 - Transform
 - Aggregate
 - Rename
 - Combine
- Data Output: Save processed data



Case Study: V-Dem Data

v-dem.net

- Indicators of democracy
- Socio-economic indicators collected from other sources
- Full dataset downloadable

Go with .csv raw data if available

Country-Year: V-Dem Full+Others version 14

All 500 V-Dem indicators and 245 indices + 57 other indicators from other data sources. For R users, we recommend to install our [vdemdata R package](#) which includes the most recent V-Dem dataset and some useful functions to explore the data.

Download

* Required fields

Email

haohan@hku.hk

Gender

- ☐ Woman
☐ Man
☐ Non-binary/non-conforming
☐ Prefer not to say

Newsletter

☐ Add my email to our mailing list and get information about updates, new revisions and more

Privacy

☒ * I approve the processing of personal data in accordance with the V-Dem privacy policy

Format *

- ☐ STATA
☒ CSV

Aside 1: Other cross-national databases

Political/ socioeconomic data

- Quality of Government: <https://www.gu.se/en/quality-government/qog-data/data-downloads>
- World Bank: <https://databank.worldbank.org/databases>
- United Nations: <https://databank.worldbank.org/databases>

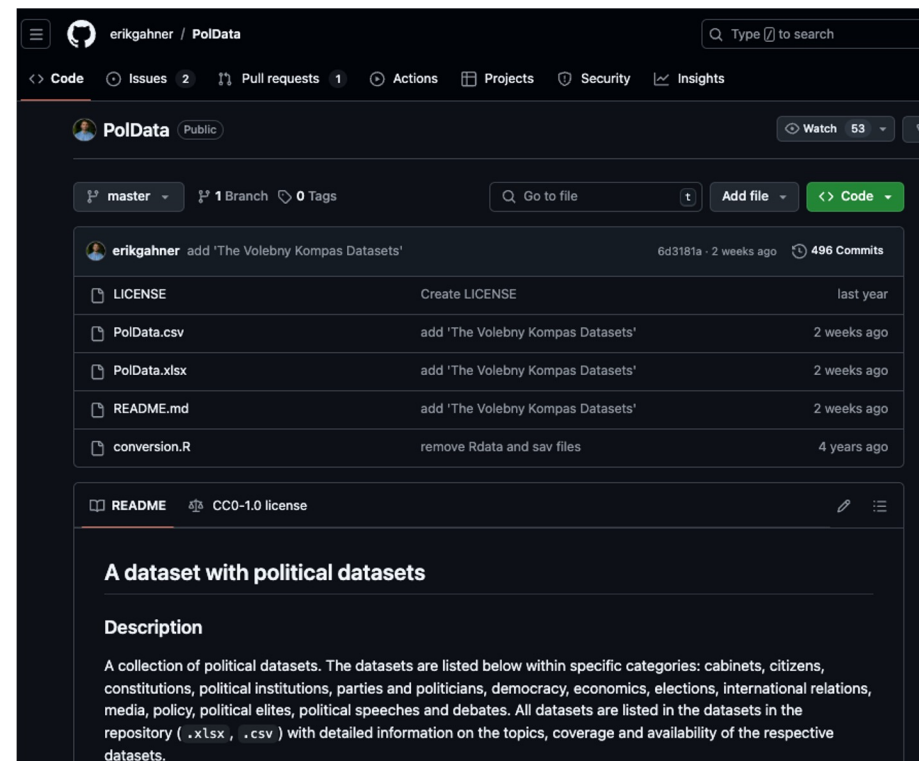
Opinion data

- World Value Survey: <https://www.worldvaluessurvey.org/>
- Global Barometer Survey: <https://www.globalbarometer.net/>

Aside 2: a comprehensive list of all political databases

Check it out!

<https://github.com/erikgahner/PolData>





Data Input and Data Output



Unbox the V-Dem dataset

- Decompress the downloaded Zip folder (24 Megabytes)
- Look at the documentations (in .pdf files) and dataset (in .csv)
- Upload to your Google Drive folder



Give Colab Access to Data

- Your raw data file is stored in your Google Drive
- To read these data files, you must first give Google Colab access to your Google Drive

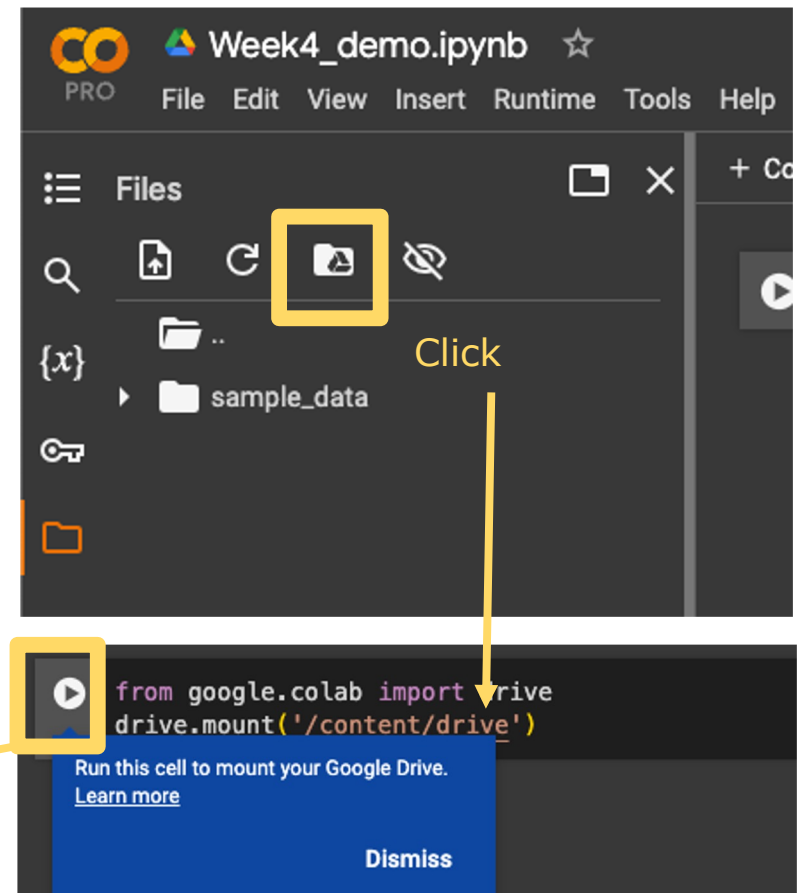
Select what **Google Drive for desktop** can access

☒ Select all

 See, edit, create, and delete all of your Google Drive files. [Learn more](#) ☒

 View the photos, videos and albums in your Google Photos. [Learn more](#) ☒

☐ Retrieve Mobile client configuration and experimentation. [Learn more](#) ☒

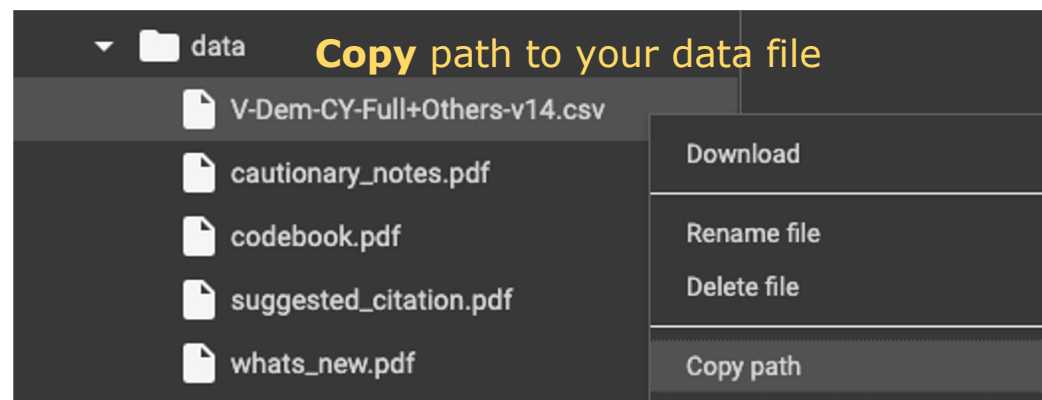
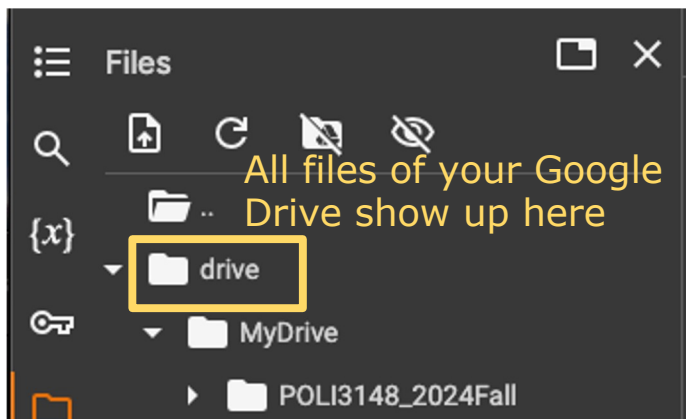


The screenshot shows the Google Colab interface for a file named 'Week4_demo.ipynb'. The 'Files' sidebar on the left contains a folder named 'sample_data'. A yellow box highlights the 'Connect to Google Drive' icon (a folder with a cloud) in the sidebar, with a yellow arrow pointing to it and the text 'Click'. Below the sidebar, a code cell contains the following Python code:

```
from google.colab import drive
drive.mount('/content/drive')
```

A yellow box highlights the play button icon to the left of the code cell. A yellow arrow points from this box to the 'Dismiss' button on a blue notification banner at the bottom of the code cell. The banner contains the text: 'Run this cell to mount your Google Drive. [Learn more](#)'.

Give Colab Access to Data con'd



Good practice re raw data storage

- Document your data sources and information of retrieval
- Organize your data files (don't throw them to random places!)
- Keep documentation (codebook, user manuals etc.)
- Always keep the raw data **unaltered**
- Collaborative project:
 - Work on the same copy of data.
 - Tip for Google Drive-based collaboration
 - Non-owners should add “shortcuts” of the folder
 - Trick: all members add a shortcut to the folder at the root folder ⇒ Everyone can run everyone else's code.

Meet the Pandas Python library

- A Python library contains an army of “minions”
- Libraries are tools that data scientists in the Python community develop to help us get work done quickly
- The **Pandas** library specializes in data wrangling and cleaning

To load the pandas library onto Python

```
import pandas
```

To load the pandas library and refer to it in your program with a (shorter) nickname

```
import pandas as pd
```



Data Input

Read a data file in .csv format

```
df = pd.read_csv("[path to data]")
```

What is a .csv file?

"csv" = comma-separated values

[Comma-separated values - Wikipedia](#)

Look up a similar format yourself: **.tsv**

Comma-separated values



```
fname, lname  
nancy, davo  
erin, bora  
tony, rapha  
...
```

```
company,surname,forename  
Foo Tech,Jones,Alice  
Top Bar Hardware,Smith,Bob  
Quxcorp,Garcia,Carlos
```

A simple CSV file listing three people and the companies they work for

A quick overview of the data

How big is the dataset? What kinds of information are available?

`df.shape`

`df.columns.tolist()`

How we call things in the data

- Columns: variables
 - Rows: records, observations
-

A comprehensive overview of the data

Check the data types of each column

df.dtypes

Get the summary statistics of ALL *numeric* (*int*, *float*, *boolean* types) columns

df.describe()

Data Output: Export data overview

Want to take a closer look of the summary statistics? Output results of **df.dtypes** and **df.describe()**

Steps:

1. Assign results to Python objects

```
sum_dtypes = df.dtypes  
sum_describe = df.describe()
```

2. Save to .csv files

```
sum_dtypes.to_csv()  
sum_describe.to_csv()
```

3. * You may save them to other formats, try **to_excel()**
4. Open them in Google Drive (or you can download them to examine if you want)

Good practice re storage of processed data

As you have your first processed data, here are some reminders re good practice:

- Separately store raw and processed data
 - NEVER overwrite raw data with processed data
 - Assign informative names to files of processed data, preferably similar to the objects' names in your Python program
 - Saving intermediate outputs is good
-

Advanced: JSON for data output and input

JSON [JSON - Wikipedia](#)

Let me demonstrate what it looks like by saving our overview data in alternative formats

```
sum_describe.to_json()
```

Can you read the output files back to Python?

```
sum_describe_read = pd.read_json("[PATH TO DATA]")
```



Data Processing



Case: China's economic development

10.3.3 GDP (E) (e_gdp)

Additional versions: *_sd

Clarification: Point estimate from latent variable model of Gross Domestic Product based on a number of sources. For details, see the citation.

Source(s): Consult the citation for the sources used.

Data release: 12-14.

Citation: Fariss *et al.* (2021)

Years: 1789-2019

10.3.4 GDP per capita (E) (e_gdppc)

Additional versions: *_sd

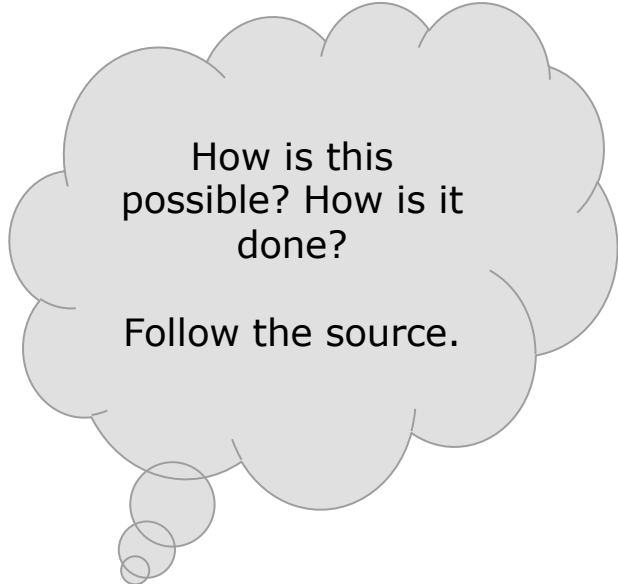
Clarification: Point estimate from latent variable model of Gross Domestic Product Per Capita based on a number of sources. For details, see the citation.

Source(s): Consult the citation for the sources used.

Data release: 12-14.

Citation: Fariss *et al.* (2021)

Years: 1789-2019



How is this possible? How is it done?

Follow the source.

Fariss, C., Anders, T., Markowitz, J. & Barnum, M. (2021), 'Replication Data for: New Estimates of Over **500** Years of Historic GDP and Population Data'. **URL:** <https://doi.org/10.7910/DVN/DC0ING>

Fariss, C. J., Anders, T., Markowitz, J. N., & Barnum, M. (2022). New Estimates of Over 500 Years of Historic GDP and Population Data. *Journal of Conflict Resolution*, 66(3), 553-591. <https://doi.org/10.1177/00220027211054432>

Subset: The .loc() function

Subset by rows

- Subset observations from China
- Subset observations from 2000-2020

```
df.loc[df.country_name == "China"]
```

```
df.loc[df.year >= 2000]
```

```
df.loc[(df.year >= 2000) & (df.year <= 2020)]
```

```
df.loc[(df.country_name == "China") & (df.year >= 2000) & (df.year <= 2020)]
```

Subset: The .loc() function

Keep economic indicators (GDP)

- GDP (e_gdp)
- GDP per capita (e_gdppc)

```
df.loc[:, ['country_name', 'year', 'e_gdp', 'e_gdppc']]
```

Explaining the syntax 1

`df.loc[row-wise subsetting criteria, column-wise subsetting criteria]`

“:” means SELECT ALL

- `df.loc[:, ['country_name', 'year', 'e_gdp', 'e_gdppc']]` means, “select all rows, and subset this list of columns”
- **Q:** Wait, but when we subset row (e.g., `df.loc[df.country_name == "China"]`), we used no “:”
- **A:** The above expression is an abbreviated version of `df.loc[df.country_name == "China", :]`.
 - Pandas allow us to omit the subsetting conditions regarding columns if subset by row only
 - **However**, if we subset by column only, we cannot omit the subsetting condition regarding rows.

Explaining the syntax 2

What are the inputs of the selection criteria

- Column/ row names, put in a list. E.g., ['country_name', 'year', 'e_gdp', 'e_gdppc']
- A list/ array/ column of booleans (True/ False). **True** means the row/ column should be included, **False** means the the row/ column should be excluded.

Note: It should have the same length as the dimension

Subset: Simultaneously subset by row and column

Request

- China
- 2000-2020
- Economic data

```
df.loc[(df.country_name == "China") & (df.year >= 2000) & (df.year <= 2020),  
       ['country_name', 'year', 'e_gdp', 'e_gdppc']]
```

Good Practice re Subsetting

- Subset step by step. Don't do all operations in one command.
- Use comments to document the subsetting operation
- Save the subset of interest for your data analysis
 - Avoid repeatedly clean and wrangle the data
 - Use informative file names
 - Keep the original/ raw data, even if you don't think you need them any more.
E.g., if you conduct a study on China and do the example operations for subset, you are strongly recommended to keep the global data

Transform

Example: Examine China's economic performance from 1949 to 2019 under different administration

Preparation: Subset China's economic data 1949-2019

Transform 1

Transform a numeric variable to another numeric variable:

- GDP in thousands (divide e_gdp by 1000)

```
df_china_econ['gdp_1000'] = df_china_econ.e_gdp / 1000
```

or

```
df_china_econ['gdp_1000'] = df_china_econ['e_gdp'] / 1000
```

To refer to an existing column, you can use either **df['column_name']** or **df.column_name**

To create a column, you can use only **df['column_name']**

Transform 2

Transform a numeric variable to another numeric variable:

- Take the logarithm of GDP

*You need to import the library **numpy** for this*

```
import numpy as np  
df_china_econ['gdp_log'] = np.log(df_china_econ.e_gdp)
```

** The **numpy** library will be our close friend. Check it out: <https://numpy.org/>*

Transform 3

- Transform a numeric variable to a binary variable
 - Years under the Xi administration: 2013 - 2023

```
df_china_econ['admin_xi'] = (df_china_econ.year >= 2013) & (df_china_econ.year <= 2023)
```

By the way, how to delete a column?

```
df_china_econ = df_china_econ.drop(columns = ['admin_xi'])
```

You can drop multiple columns at a time, just add names to the list, for example

```
df_china_econ = df_china_econ.drop(columns = ['gdp_1000', 'gdp_log', 'admin_xi'])
```

(Advanced) Transform 4

- Transform a numeric variable to a categorical variable indicating the leaders of the Chinese regime of the year
 - Mao: 1949-1976
 - Hua-Hu-Zhao: 1976-1989
 - Jiang: 1989-2002
 - Hu: 2002-2013
 - Xi: 2013-2023

How? Use the subsetting **.loc()** function in a different way

(Advanced) Transform 4 (con'd)

```
df_china_econ['admin'] = None # Initiate a column named admin with None type
```

```
df_china_econ.loc[(df_china_econ.year >= 1949) & (df_china_econ.year < 1976), 'admin'] = "Mao"
```

```
df_china_econ.loc[(df_china_econ.year >= 1976) & (df_china_econ.year < 1989), 'admin'] = "Hua-Hu-Zhao"
```

```
df_china_econ.loc[(df_china_econ.year >= 1989) & (df_china_econ.year < 2002), 'admin'] = "Jiang"
```

```
df_china_econ.loc[(df_china_econ.year >= 2002) & (df_china_econ.year < 2013), 'admin'] = "Hu"
```

```
df_china_econ.loc[(df_china_econ.year >= 2013) & (df_china_econ.year <= 2023), 'admin'] = "Xi"
```

Simple Aggregate 1

Aggregate one column

What was the average, best and worst economic performance?

- Overall
- Under each administration

```
df_china_econ.gdp_1000.agg(['mean', 'min', 'max', 'std'])
```

```
df_china_econ.groupby('admin').gdp_1000.agg(['mean', 'min', 'max', 'std'])
```

Simple Aggregate 2

Aggregate multiple columns

What was the average, worst, and best economic outcome in terms of *both* GDP and GDP per capita?

```
df_china_econ.agg({'gdp_1000': ['mean', 'min', 'max'], 'e_gdppc': ['mean', 'min', 'max', 'std']})
```

```
df_china_econ.groupby('admin').agg({'gdp_1000': ['mean', 'min', 'max'], 'e_gdppc': ['mean', 'min', 'max', 'std']})
```


Simple Aggregate 2 con'd (alternative, HC's preferred approach)

```
df_china_econ.groupby('admin').agg(  
    gdp_mean = ('gdp_1000', 'mean'),  
    gdp_min = ('gdp_1000', 'min'),  
    gdp_max = ('gdp_1000', 'max'),  
    gdp_std = ('e_gdppc', 'std'),  
    gdppc_mean = ('e_gdppc', 'mean'),  
    gdppc_min = ('e_gdppc', 'min'),  
    gdppc_max = ('e_gdppc', 'max'),  
    gdppc_sd = ('e_gdppc', 'std')  
)
```

```
.reset_index()
```

**Hierarchical indexing
brings more troubles than
help. Don't use it!**

Aside: Subset observation of the max or min

Keep only the observations that has the best and worst economic performance under each administration

```
df_china_econ.loc[df_china_econ.groupby('admin').gdp_1000.idxmin()]
```

```
df_china_econ.loc[df_china_econ.groupby('admin').gdp_1000.idxmax()]
```

New task

We want to **compare** the economy of China and Japan from 1949 to 2019

Prep: Create a new subset named **df_japan_econ**

```
df_japan_econ = df.loc[(df.country_name == "Japan") & (df.year >= 1949) &
(df.year <= 2019), ['country_name', 'year', 'e_gdp', 'e_gdppc']
```

Combine the two datasets: Horizontal

- Rename columns
- Merge the datasets by column using **year** as the identifier

Rename columns

```
df_china_econ_h = df_china_econ[['year', 'e_gdp', 'e_gdppc']]
```

```
.rename(columns = {'e_gdp': 'gdp_china', 'e_gdppc': 'gdppc_china'})
```

```
df_japan_econ_h = df_japan_econ[['year', 'e_gdp', 'e_gdppc']]
```

```
.rename(columns = {'e_gdp': 'gdp_japan', 'e_gdppc': 'gdppc_japan'})
```

Combine the two datasets: Horizontal (con'd)

Join: According to Kaggle (needs to set index to the match key)

```
df_china_econ_h = df_china_econ_h.set_index('year')  
df_japan_econ_h = df_japan_econ_h.set_index('year')  
df_cn_jp = df_china_econ_h.join(df_japan_econ_h)
```

Merge: Simpler method

```
df_cn_jp = pd.merge(df_china_econ_h, df_japan_econ_h, on = 'year')
```

Combine the two datasets: Vertical

You can also combine the two datasets vertically

```
df_china_econ_v = df_china_econ[['country_name', 'year', 'e_gdp', 'e_gdppc']]
df_japan_econ_v = df_japan_econ[['country_name', 'year', 'e_gdp', 'e_gdppc']]
pd.concat([df_china_econ_v, df_japan_econ_v])
```

Think: Why do I keep the “country_name” column for vertical combination, but remove it for horizontal combination?



In-class Exercise 3

